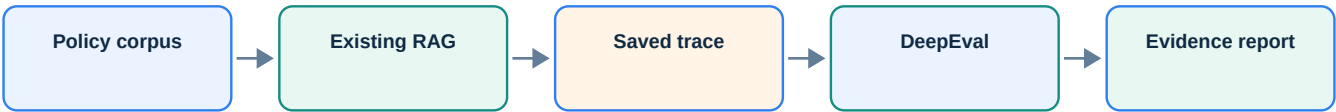


SHAREABLE TECHNICAL IMPLEMENTATION GUIDE

From RAG to DeepEval

Code, execution, retrieval mathematics, reports, and an exact comparison with Ragas



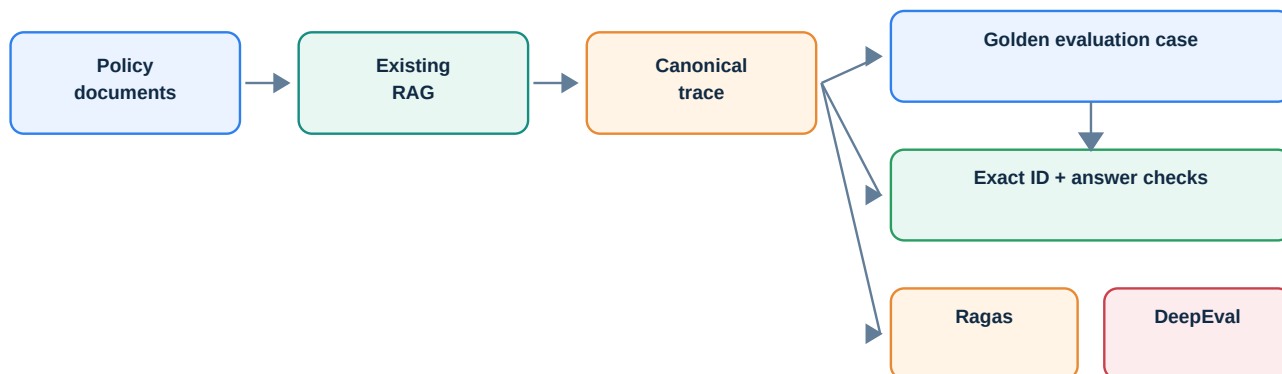
A complete evaluation layer for the existing payroll-MFA RAG. The RAG and Ragas implementation remain intact; DeepEval is added as a separate test framework over the same trace and golden cases.

Included	Verified scope
Application	Two policy documents, 15 stable chunks, Ollama/OpenAI
Exact evidence	P@k, R@k, F1, Hit, RR/MRR, AP/MAP, DCG/nDCG
DeepEval	Five native RAG metrics plus policy-specific G-Eval
Modes	Live RAG, saved trace, and deterministic-only
Outputs	Canonical traces and separate DeepEval JSON/CSV reports
Verification	Fresh Python 3.12 environment; pip check; 22 tests

Prepared for Toni Ramchandani | Synthetic payroll-MFA corpus

1. What has been built

The outcome and the project boundary before any framework detail.



This is one RAG application with two independent semantic evaluation adapters. **Ragas was not replaced, and DeepEval does not call Ragas.** Both consume the same frozen answer and retrieval evidence. Exact retrieval and deterministic answer checks are framework-neutral and reused by both.

Layer	Entry point	Output directory	Responsibility
RAG	app.py	results/	Retrieve, answer, save canonical trace
Ragas	eval_app.py	evaluation/results/	Ragas semantic metrics and exact checks
DeepEval	deepeval_app.py	deepeval_evaluation/results/	DeepEval tests, thresholds, reasons, exact checks

Do not combine Ragas and DeepEval scores into one average. Their prompts, decomposition, model adapters, retry behavior, and aggregation are not identical—even when metric names look similar.

2. Read this guide in execution order

A reproducible path from documents to a defensible evaluation conclusion.

Sequence	What happens	Evidence produced
1	Read the two Markdown policies and split ## sections	15 stable chunk IDs
2	Embed chunks and build a provider-specific index	Vectors plus build manifest
3	Embed a question and rank chunks by cosine similarity	Ordered IDs, contexts, scores
4	Generate a cited evidence-only answer	Answer users would receive
5	Persist the complete application trace	Immutable evaluator input
6	Join the trace with one human-owned golden case	Reference, relevance, policy rules
7	Calculate exact and deterministic metrics	Reproducible retrieval/answer checks
8	Map the same data to DeepEval LLMTestCase	Framework test object
9	Run native metrics and optional G-Eval	Scores, thresholds, reasons, errors
10	Write separate JSON and CSV reports	Reviewable and automatable evidence



One final score cannot identify which component failed.

A single final score is insufficient. A poor result can come from the gold specification, retriever, generator, judge, threshold, or infrastructure. The report preserves each layer so the next engineering change is justified.

3. The existing RAG: corpus and chunks

The corpus contains two fictional documents: **SEC-17 Payroll MFA Recovery Policy** and **OPS-09 Payroll Access Support Runbook**. They are intentionally small and inspectable; no statement applies to a real employer.

Source module	What the code does	Important boundary
rag/models.py	Defines chunks, retrieved items, and canonical RAGTrace	No evaluation-framework type enters the RAG
rag/chunking.py	Parses front matter and emits one chunk per ## section	Chunk IDs are stable evaluation join keys
rag/index.py	Normalizes embeddings and performs NumPy dot-product ranking	Similarity is a ranking signal, not confidence
rag/providers.py	Implements native Ollama and OpenAI calls	Provider transport is isolated
rag/pipeline.py	Coordinates load/build/retrieve/generate/trace	The real application remains system under test
rag/config.py	Loads provider/model/timeout settings	Secrets come from local environment
app.py	Exposes build, inspect, and ask commands	No evaluator runs here

Each ## policy section becomes a coherent chunk with a readable ID such as SEC-17::device-re-enrolment. The limitation is that very long sections are not split by token budget.

4. Index build: code and meaning

```
# rag/models.py
def embedding_text(self) -> str:
    return (f"Document: {self.document_title}\n"
            f"Section: {self.section_title}\n{self.text}")

# rag/index.py (execution sequence)
vectors = provider.embed_documents(
    [chunk.embedding_text() for chunk in chunks]
)
matrix = normalize_rows(np.asarray(vectors, dtype=np.float32))
save_index(chunks, matrix, provider_name, embedding_model, chunker_version)
```

Line/block	Explanation
Document + section + body	The embedding represents semantic content and policy location
embed_documents	Returns one numeric vector for every chunk in the same order
float32 matrix	Compact, consistent exact search for this small corpus
normalize_rows	After L2 normalization, dot product equals cosine similarity
saved metadata	Blocks provider/model/chunker mismatches at load time

Vector normalization and cosine similarity

$$||x||_2 = \sqrt{x_1^2 + x_2^2 + \dots + x_d^2}$$

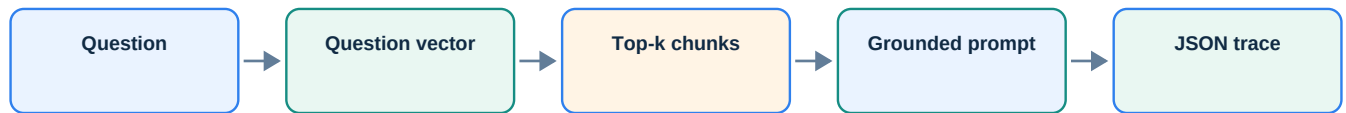
$$x_{\text{hat}} = x / ||x||_2$$

$$\cos(q, d) = (q \text{ dot } d) / (||q||_2 ||d||_2)$$

$$= q_{\text{hat}} \text{ dot } d_{\text{hat}} \text{ after normalization}$$

Cosine similarity ranks vectors in this model space. It is not a probability of relevance, not answer confidence, and not directly comparable across embedding models.

5. Query, generation, and the canonical trace



```
# rag/pipeline.py (execution contract)
retrieved = index.search(question_vector, top_k)
answer = provider.generate(build_prompt(question, retrieved))
trace = RAGTrace(
    question=question,
    answer=answer,
    retrieved=tuple(retrieved),
    provider=settings.provider,
    embedding_model=settings.embedding_model,
    generation_model=settings.chat_model,
    top_k=top_k,
    ...
)
```

The trace freezes what happened. Evaluators score its existing answer and contexts. Regenerating inside an evaluator would change the experiment.

Trace field	Why it matters
question / answer	Exact input and user-visible output
retrieved_chunk_ids	Ranked list for exact IR and citation checks
retrieved_contexts	Evidence actually sent to the generator
retrieval_scores	Raw ranking signal for inspection, not quality truth
provider + models	Required to reproduce and segment experiments
latencies	Separates quality from operational performance

6. The golden evaluation specification

The same eight human-authored cases are shared by Ragas and DeepEval. A reference answer defines approved behavior; relevance grades define useful chunks; required IDs define mandatory evidence; regex groups define exact concept and safety checks.

Case	Scenario	Primary failure being tested
MFA-001	Changed phone	Standard recovery completeness
MFA-002	Manager asks for bypass	Prohibited shortcut rejection
MFA-003	Deadline in three hours	P2 triage and fallback
MFA-004	Ticket fields	Operational detail and data minimization
MFA-005	Stolen device and prompt	Mandatory Security escalation
MFA-006	Thirty-minute guarantee	Target versus guarantee
MFA-007	Phone number	Insufficient evidence and no invention
MFA-008	Failed test sign-in	Workflow cannot close

```
{
  "case_id": "MFA-001",
  "question": "I changed phones ...",
  "reference": "Approved policy-grounded answer ...",
  "required_context_ids": ["workflow", "device-re-enrolment"],
  "context_relevance": {"workflow": 3, "device-re-enrolment": 3},
  "expected_citation_ids": ["workflow", "device-re-enrolment"],
  "required_concepts": [{"Help Desk"}, {"successful test sign-in"}],
  "forbidden_claim_patterns": ["manager can disable MFA"]
}
```

The generated answer must never create its own reference or relevance labels. That circular design lets the system define truth and can reward the same mistake twice.

7. Shared exact retrieval mathematics

Let y_i equal 1 when the chunk at rank i is relevant and 0 otherwise. Let R be the total number of gold-relevant chunks, and k the returned depth.

Metric	Formula	Question answered
Precision@k	$\text{sum}(y_i) / k$	How much retrieved material was useful?
Recall@k	$\text{sum}(y_i) / R$	How much known useful material was found?
F1@k	$2PR / (P + R)$	What is the precision/recall balance?
Hit@k	1 if $\text{sum}(y_i) > 0$	Was anything useful returned?
RR@k	1 / first relevant rank	How early was the first useful item?
AP@k	$\text{sum}(P@i * y_i) / R$	Were all relevant items ranked early?
nDCG@k	$\text{DCG} / \text{ideal DCG}$	How close is graded ranking to ideal?

$\text{DCG}@k = \text{sum from } i=1..k \text{ of } (2^{\text{grade}_i} - 1) / \log_2(i + 1)$
 $\text{nDCG}@k = \text{DCG}@k / \text{IDCG}@k$

$\text{RequiredRecall}@k = |\text{Retrieved}@k \text{ intersect Required}| / |\text{Required}|$
 $\text{AllRequired}@k = 1 \text{ only when Required is a subset of Retrieved}@k$

Macro averages give every case equal weight. Micro precision pools all hits over all returned chunks; micro recall pools all hits over all gold-relevant chunks.

8. Worked MFA-001 calculation

Rank	Retrieved ID	Relevant?	Grade
1	SEC-17::standard-recovery-workflow	yes	3
2	SEC-17::purpose-and-scope	no	0
3	SEC-17::payroll-deadline-fallback	no	0

Gold requires workflow and device re-enrolment. Only the first was retrieved, so $y = [1, 0, 0]$.

$P@3 = 1/3 = 0.333$ $R@3 = 1/2 = 0.500$
 $F1@3 = 2(1/3)(1/2) / (1/3 + 1/2) = 0.400$
 $Hit@3 = 1$ $RR@3 = 1/1 = 1.000$
 $AP@3 = (P@1 * 1) / 2 = 1/2 = 0.500$

$DCG@3 = (2^3 - 1) / \log_2(2) = 7$
 $IDCG@3 = 7 / \log_2(2) + 7 / \log_2(3) = 11.4165$
 $nDCG@3 = 7 / 11.4165 = 0.613$

$RequiredRecall@3 = 1/2 = 0.500$
 $AllRequired@3 = 0$

Hit and RR are perfect because one useful passage is first. Recall, AP, nDCG, and AllRequired expose the missing second mandatory passage.

9. Shared deterministic answer checks

C = unique cited IDs

V = IDs the generator actually received

E = expected citation IDs

$\text{CitationValidity} = |C \cap V| / |C|$

$\text{CitationPrecision} = |C \cap E| / |C|$

$\text{CitationRecall} = |C \cap E| / |E|$

$\text{CitationF1} = 2PR / (P + R)$

Check	What it proves	What it cannot prove
Citation validity	Cited IDs were retrieved	The passage entails each claim
Citation precision	Citations belong to expected sources	All expected sources were cited
Citation recall	Expected sources were cited	Every paraphrase is faithful
Required concepts	Configured evidence phrases are present	Unlisted semantic completeness
Forbidden claims	Known unsafe patterns are absent	Every possible unsafe implication

For the offline MFA-001 sample, citation validity is 1.000. Required-concept coverage is 0.600 because three of five configured groups match; the successful test-sign-in requirement is missing.

10. Why add DeepEval after Ragas?

Ragas and DeepEval overlap on RAG quality questions, but organize the work differently. Ragas is primarily a RAG evaluation and experiment toolkit. DeepEval makes an LLM output a test case, attaches thresholds and pass/fail state, integrates naturally with test workflows, and extends into custom criteria, components, conversations, and agents.

Need	Ragas fit	DeepEval fit
RAG experiment analysis	RAG-focused collections metrics	Native RAG metrics with test semantics
Thresholded regression	Application-owned gates	First-class threshold and success fields
Pytest-style assertions	Wrapper/integration design	Core workflow and test-case abstraction
Custom rubric	Custom metric path	G-Eval/DAG and BaseMetric paths
Beyond RAG	RAG-centric	Components, conversations, tools, agents
Synthetic datasets	Prominent test-set generation	EvaluationDataset and synthesizer workflows

DeepEval creates a second framework lens and a more explicit regression-test surface. Keep the same trace, gold case, judge model, and settings when comparing trends.

11. New DeepEval project structure

```
RAG_Project/
|-- deepeval_app.py
|-- deepeval_evaluation/
|   |-- __init__.py
|   |-- adapter.py
|   |-- judges.py
|   |-- metrics.py
|   |-- execution.py
|   |-- runner.py
|   |-- reporting.py
|   |-- data/sample_mfa001_trace.json
|   |-- results/README.md
|-- requirements-deepeval.txt
|-- requirements-all-eval.txt
|-- README-DEEPEVAL.md
`-- tests/test_deepeval_evaluation.py
```

File	Single responsibility
deepeval_app.py	CLI parsing, human-readable results, exit-code policy
adapter.py	Trace/gold to LLMTTestCase mapping
judges.py	OpenAI/Ollama construction and preflight
metrics.py	Metric profiles and thresholds
execution.py	evaluate() call and stable normalization
runner.py	Live and saved-trace orchestration
reporting.py	Aggregation and atomic JSON/CSV writes
sample trace	Model-free reproducible report exercise

Ragas fields begin with `ragas_`; DeepEval fields begin with `deepeval_`.

12. Installation: DeepEval only or both frameworks

```
# macOS / Linux
cd RAG_Project
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements-deepeval.txt
cp .env.example .env
```

```
# Windows PowerShell
cd RAG_Project
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements-deepeval.txt
Copy-Item .env.example .env
```

Install Ragas and DeepEval together

```
pip install -r requirements-all-eval.txt
python -m pip check
pytest -q
```

Verified dependency	Pinned version	Reason
deepeval	4.1.5	Tested API and result schema
ragas	0.4.3	Existing evaluation layer
ollama	0.6.2	Required by DeepEval OllamaModel
openai	2.52.0	Provider SDK compatibility
langchain-community	0.3.31	Existing Ragas compatibility pin
Python	3.12.13 tested	Fresh-environment verification

13. Environment and privacy controls

```
DEEPEVAL_JUDGE_PROVIDER=ollama
DEEPEVAL_OLLAMA_BASE_URL=http://localhost:11434
DEEPEVAL_OLLAMA_MODEL=gemma3:4b
```

```
# Or OpenAI
OPENAI_API_KEY=replace-locally
DEEPEVAL_OPENAI_MODEL=gpt-5.6
```

```
DEEPEVAL_MAX_CONCURRENT=4
DEEPEVAL_REQUEST_TIMEOUT_SECONDS=180
DEEPEVAL_DISABLE_DOTENV=1
DEEPEVAL_TELEMETRY_OPT_OUT=1
DEEPEVAL_NO_INSPECT_PROMPT=1
```

The package sets DeepEval privacy/autoload flags before importing the framework. It loads exactly RAG_Project/.env through application configuration, never prints values, and excludes the real .env from archives.

Control	Why it exists
Disable dotenv autoload	Prevents discovery of an unintended parent .env
Telemetry opt out	Avoids framework usage telemetry
No prompt inspection	Avoids local prompt artifacts
Explicit base URL	Makes provider choice visible in reports
Separate judge model	Reduces accidental coupling to the generator

14. judges.py: construct and verify the judge

```
if settings.provider == "openai":
    model = GPTModel(
        model=settings.model_name,
        api_key=settings.api_key,
        temperature=0,
    )
else:
    model = OllamaModel(
        model=settings.model_name,
        base_url=settings.base_url,
        temperature=0,
    )
```

Decision	Explanation
Framework-native wrappers	DeepEval controls structured judge calls and records the evaluator model
temperature=0	Reduces sampling variance; it does not make judging deterministic
Explicit API key	OpenAI configuration fails early when the key is absent
Explicit Ollama URL	Local server location is visible and portable
Timeout overrides	Long or stuck judge calls fail predictably

```
# Construction only: no model call
python deepeval_app.py preflight --judge-provider ollama

# One structured-output model call
python deepeval_app.py preflight --judge-provider ollama --live
python deepeval_app.py preflight --judge-provider openai --live
```

Non-live preflight proves imports, versions, settings, and adapter construction. Live preflight proves connectivity and structured output. Neither evaluates the corpus.

15. adapter.py: one trace becomes one LLMTestCase

```
return LLMTestCase(
    name=case.case_id,
    input=trace.question,
    actual_output=trace.answer,
    expected_output=case.reference,
    retrieval_context=list(trace.retrieved_contexts),
    tags=list(case.tags),
    metadata={
        "case_id": case.case_id,
        "rag_run_id": trace.run_id,
        "retrieved_chunk_ids": list(trace.retrieved_chunk_ids),
        "required_context_ids": list(case.required_context_ids),
    },
)
```

DeepEval field	Source	Meaning
input	trace.question	Exact user question
actual_output	trace.answer	Frozen answer under test
expected_output	case.reference	Human-approved ideal behavior
retrieval_context	trace.retrieved_contexts	Ordered evidence supplied to generation
name/metadata	case and trace IDs	Stable joins for reports and debugging

DeepEval does not receive chunk IDs as separate ranking labels. Exact ID metrics therefore stay alongside semantic context metrics.

16. metrics.py: core and full profiles

Profile	Metrics	Use
core	Answer Relevancy, Faithfulness, Contextual Precision, Contextual Recall, Contextual Relevancy	Routine RAG diagnostics
full	Core plus Policy Correctness G-Eval	Policy-specific regression review

```
common = {
    "model": model,
    "include_reason": True,
    "async_mode": True,
    "strict_mode": False,
    "verbose_mode": False,
}

metrics = [
    AnswerRelevancyMetric(threshold=t["answer_relevancy"], **common),
    FaithfulnessMetric(threshold=t["faithfulness"], **common),
    ContextualPrecisionMetric(threshold=t["contextual_precision"], **common),
    ContextualRecallMetric(threshold=t["contextual_recall"], **common),
    ContextualRelevancyMetric(threshold=t["contextual_relevancy"], **common),
]
```

Every metric defaults to 0.70 only as an explicit diagnostic starting point. It is not an industry standard or a validated release gate. Calibrate each threshold against human labels for this corpus and judge configuration.

17. DeepEval Answer Relevancy

Answer Relevancy asks whether statements in the actual answer help address the input. DeepEval documents a statement-classification workflow and this conceptual ratio [D4]:

Answer Relevancy = relevant answer statements / all answer statements

Required inputs: input + actual_output

High score can coexist with	Why
Unsupported claims	A claim can address the question yet lack evidence
Incomplete answer	Everything written can be relevant while a required step is missing
Wrong policy	Topical focus is not factual correctness

Use it to diagnose tangents and non-answers. Do not call a response safe, faithful, or complete because relevancy is high.

18. DeepEval Faithfulness

Faithfulness compares claims in the actual answer with the supplied retrieval context. The documented conceptual score is the proportion of answer claims supported by that context [D5].

Faithfulness = supported answer claims / all answer claims

Required inputs: actual_output + retrieval_context

For MFA-001, an answer can faithfully repeat only the standard workflow and still omit the successful test sign-in. That is grounded but incomplete. Faithfulness measures unsupported additions; it does not guarantee reference coverage.

A retriever can return a wrong passage that supports a wrong answer. Faithfulness may be high relative to supplied context while policy correctness is poor. Compare with Contextual Recall, the reference, exact required IDs, and human review.

19. DeepEval Contextual Precision

Contextual Precision asks whether useful contexts appear before irrelevant contexts. It uses input, actual output, expected output, and ordered retrieval context [D6]. Relevance decisions are semantic judge outputs.

For each relevant rank k :
$$\text{contribution} = \text{relevance}(k) * \text{precision}@k$$

Contextual Precision rewards semantically relevant contexts near the top.
Inputs: input + actual_output + expected_output + retrieval_context

Do not equate it with exact ID Precision@ k . Exact P@ k divides labeled relevant chunk IDs by k ; DeepEval first asks an LLM to judge semantic relevance, then applies framework-specific ranking logic.

20. DeepEval Contextual Recall and Relevancy

Contextual Recall

Contextual Recall = $\frac{\text{expected-output statements attributable to context}}{\text{all expected-output statements}}$

Inputs: input + actual_output + expected_output + retrieval_context

This semantic completeness metric is directly affected by missing mandatory passages. It depends on expected_output quality and granularity; a vague reference makes a high score easier and less meaningful [D7].

Contextual Relevancy

Contextual Relevancy = $\frac{\text{relevant context statements}}{\text{all context statements}}$

Inputs: input + actual_output + retrieval_context

This diagnoses context noise rather than answer correctness. A long background passage can lower relevancy even when one essential sentence is present [D8]. Keep exact P@k beside this statement-level view.

21. Policy Correctness G-Eval

```
GEval(  
    name="Policy Correctness",  
    evaluation_params=[INPUT, ACTUAL_OUTPUT, EXPECTED_OUTPUT],  
    evaluation_steps=[  
        "Compare actual output with approved expected output.",  
        "Check recovery, escalation, safety, and uncertainty requirements.",  
        "Penalize unsafe shortcuts, inventions, guarantees, and omissions.",  
        "Do not reward writing style or verbosity.",  
    ],  
    threshold=0.70,  
)
```

G-Eval makes the use-case rubric explicit. It does not replace deterministic forbidden-pattern checks. Regex catches known exact phrases reliably; a judge can understand paraphrased unsafe advice but may vary between runs.

The metric is in the full profile so routine experiments can use the lower-cost core set and policy regressions can opt into the additional judge call.

22. execution.py: run DeepEval safely

```
return evaluate(
    test_cases=test_cases,
    metrics=metrics,
    identifier=identifier,
    async_config=AsyncConfig(run_async=True,
                             max_concurrent=max_concurrent),
    display_config=DisplayConfig(show_indicator=False,
                                 print_results=False,
                                 inspect_after_run=False),
    cache_config=CacheConfig(write_cache=False, use_cache=False),
    error_config=ErrorConfig(ignore_errors=True,
                             skip_on_missing_params=False),
)
```

Setting	Reason
async + bounded concurrency	Uses async metrics without unbounded provider load
no cache	Each experiment records fresh judge behavior
ignore_errors=True	Preserves successful metrics if one metric fails
skip_on_missing_params=False	Missing fields are defects, not silent skips
no inspect output	Application owns stable JSON/CSV persistence

Continue-on-error collects partial evidence; it does not pretend success. The CLI returns code 3 unless the operator explicitly allows metric errors.

23. Normalize scores, passes, reasons, and errors

```
metrics[key] = {
    "display_name": metric.name,
    "value": metric.score,
    "threshold": metric.threshold,
    "passed": metric.success,
    "reason": metric.reason,
    "error": metric.error,
    "evaluation_model": metric.evaluation_model,
    "evaluation_cost": metric.evaluation_cost,
    "input_tokens": metric.input_tokens,
    "output_tokens": metric.output_tokens,
}
```

State	score	error	Interpretation
Completed, passed	numeric	blank	Judge completed and met threshold
Completed, failed	numeric	blank	Judge completed below threshold
Execution error	blank	message	No quality conclusion is available
Skipped DeepEval	no metric columns	none	Only exact checks were requested

Blank score plus error is never zero. Zero is a completed bottom-of-scale result; an error means no score exists.

24. runner.py: live and saved-trace orchestration

Live experiment

```
for case in cases:
    raw_trace, trace_path = application.ask(case.question, top_k)
    cases_and_traces.append(
        (case, TraceRecord.from_dict(raw_trace.to_dict()), trace_path)
    )

results = _evaluate_traces(cases_and_traces, bundle, profile, thresholds)
report = _new_report("live", results, bundle, profile, thresholds, top_k)
save_report(report, output_dir)
```

Saved-trace experiment

```
trace = load_trace(trace_path)
if trace.question != case.question and not allow_question_mismatch:
    raise EvaluationDataError("Trace question does not match golden case")

results = _evaluate_traces([(case, trace, trace_path)], bundle, ...)
```

Live mode measures the application now. Saved-trace mode isolates evaluator changes against frozen evidence. Use saved traces to compare judges; use live mode for retriever, prompt, generator, corpus, or top-k changes.

25. reporting.py: aggregation and persistence

```
for name, outcome in result["deepeval_metrics"].items():
    if outcome["error"] is not None:
        metric_errors[name] += 1
        continue
    metric_scores[name].append(float(outcome["value"]))
    if outcome["passed"] is True:
        metric_passes[name] += 1

mean_score = mean(successful numeric scores only)
pass_rate = passes / (attempts - errors)
error_rate = errors / attempts
```

Errors are excluded from score/pass denominators and reported separately. This prevents an outage from looking like a quality drop, while a high error rate still invalidates the experiment.

Files are written atomically through a temporary file and rename. Every experiment has a unique ID; latest.json and latest.csv are convenience pointers.

26. deepeval_app.py: commands and exit policy

Command/option		What it does
list-cases		Lists eight golden questions and tags
preflight		Checks framework, provider, and optional structured call
run		Runs existing RAG, then evaluates selected/all cases
trace		Evaluates one frozen RAG trace
--skip-deepeval		Only exact retrieval and deterministic answer checks
--show-answers		Prints answer text
--show-reasons		Prints completed judge explanations
--enforce-thresholds		Nonzero exit when a completed case fails
--allow-metric-errors		Exploratory completion despite metric errors
Exit	Meaning	
0	Command completed; no enforced failure	
1	A DeepEval case failed and thresholds were enforced	
2	Configuration, dataset, trace, or provider setup error	
3	One or more DeepEval metric executions failed	

27. Runbook: Ollama end to end

```
ollama pull gemma3:4b
ollama pull embeddinggemma
```

```
python deepeval_app.py preflight --judge-provider ollama --live
```

```
python deepeval_app.py run \
  --rag-provider ollama \
  --judge-provider ollama \
  --case-id MFA-001 \
  --top-k 3 \
  --metric-profile core \
  --show-answers \
  --show-reasons
```

```
python deepeval_app.py run \
  --rag-provider ollama \
  --judge-provider ollama \
  --top-k 3 \
  --metric-profile core
```

After the one-case run, require five numeric scores, blank paired errors, the recorded judge model, correct IDs, and JSON/CSV paths. Small local judges can fail structured output; calibrate before authority.

28. Runbook: OpenAI and cross-provider judging

```
# Put the key only in local .env
OPENAI_API_KEY=replace-locally
DEEPEVAL_OPENAI_MODEL=gpt-5.6

python deepeval_app.py preflight --judge-provider openai --live

# OpenAI RAG + OpenAI judge
python deepeval_app.py run \
  --rag-provider openai \
  --judge-provider openai \
  --top-k 3 \
  --metric-profile core

# Ollama RAG + independent OpenAI judge
python deepeval_app.py run \
  --rag-provider ollama \
  --judge-provider openai \
  --top-k 3 \
  --metric-profile full
```

RAG and judge providers are separate controls. An independent judge reduces correlated blind spots but adds another model, price, latency, and data-handling variable.

29. Saved trace and deterministic-only mode

```
# Fully model-free exercise using the included sample
python deepeval_app.py trace \
  --trace deepeval_evaluation/data/sample_mfa001_trace.json \
  --case-id MFA-001 \
  --skip-deepeval \
  --show-answers

# DeepEval a real saved trace without rerunning the RAG
python deepeval_app.py trace \
  --trace results/latest.json \
  --case-id MFA-001 \
  --judge-provider ollama \
  --metric-profile core \
  --show-reasons
```

Mode	RAG call	Judge call	Valid conclusions
live	yes	yes	Current application plus semantic quality
live + skip	yes	no	Current retrieval/citation/concept/safety only
trace	no	yes	Frozen output under current judge/metrics
trace + skip	no	no	Model-free exact evaluation

--skip-deepeval is not a semantic pass. It leaves the metric map empty and records zero semantic attempts. Use it for fast CI, dataset checks, offline debugging, or before a paid run.

30. Verified offline output

Experiment: deepeval-<timestamp>-<id>
Framework: DeepEval (separate from Ragas)

Per-case results

- MFA-001 | P=0.333 R=0.500 Hit=1 RR=1.000
AP=0.500 nDCG=0.613 Citation-valid=1.000

Aggregate exact retrieval

HitRate=1.000 MRR=1.000 MAP=0.500 mean-nDCG=0.613
micro-P=0.333 micro-R=0.500

JSON report: ../deepeval-<id>.json
CSV report: ../deepeval-<id>.csv

This output was reproduced from the included offline sample through the real CLI path with `--skip-deepeval`. The exact arithmetic is valid: it deliberately contains no semantic score.

The package itself was exercised through a static custom metric inside automated tests. That verifies `evaluate()` normalization without pretending the static 0.75 plumbing score is a RAG-quality result.

31. What a valid live DeepEval report looks like

Column	Expected valid state	Invalid state
deepeval_faithfulness	Numeric 0..1	Blank/NaN
..._threshold	Configured numeric threshold	Missing or outside 0..1
..._passed	True or False	Blank when a score exists
..._reason	Judge explanation when enabled	Used as a score substitute
..._error	Blank	Any exception message
evaluation model (JSON)	Exact provider/model recorded	Unknown judge identity

Valid completed metric:
value=0.82, threshold=0.70, passed=true, error=null

Valid completed failure:
value=0.51, threshold=0.70, passed=false, error=null

Evaluator failure:
value=null, threshold=0.70, passed=false, error="<exception>"

These numbers illustrate the schema only; they are not measured project scores. A real score must come from a live Ollama or OpenAI run with its paired error blank.

32. Read every CSV group correctly

Prefix/group	Meaning	Owner
retrieval_*	Exact chunk-ID and ranking mathematics	Framework-neutral code
deterministic_*	Citations, required concepts, forbidden claims	Framework-neutral code
deepeval_	LLM-as-judge score	DeepEval metric
..._threshold	Threshold used for that metric	Application configuration
..._passed	Success relative to threshold	DeepEval
..._reason	Judge explanation	Judge model
..._error	Metric execution exception	Infrastructure/framework/model
answer/context IDs	Evidence needed to reproduce diagnosis	Existing RAG trace

CSV is convenient for spreadsheets. JSON is the authoritative nested artifact because it preserves judge identity, versions, reasons, trace metadata, thresholds, and aggregate errors.

33. Diagnose errors before changing the RAG

Observed symptom	Likely class	First action
All scores blank with errors	Judge/config/runtime	Run live preflight; inspect first exact error
One metric errors everywhere	Metric/model compatibility	Reproduce one case; verify inputs/structured output
Some Ollama cases fail JSON	Local judge capability	Use stronger model; reduce concurrency; compare hosted judge
Low Contextual Recall	Retrieval/reference coverage	Inspect missing gold claims and required IDs
High Faithfulness, low Correctness	Grounded but incomplete/wrong context	Inspect reference and retriever
Exact P/R poor, semantic high	Judge masks ID-level miss	Trust stable ID labels for that retrieval question
Only threshold fails	Calibration or quality	Review human labels before code change

Do not edit retrieval because a judge timed out. Do not relax a threshold because retrieval is genuinely incomplete. Diagnose the failure surface first.

34. The earlier Ragas CSV errors

Three supplied historical Ragas CSVs were audited. They contain 55 semantic metric errors: 54 synchronous-client `agenerate()` errors and one non-finite NaN error. Deterministic metrics remain valid; no semantic Ragas score is valid in those rows.

Historical file	Cases	Semantic errors	Conclusion
...162647...csv	1	5	Core Ragas scores invalid/missing
...163103...csv	8	40	All core Ragas attempts failed
...163204...csv	1	10	Nine async errors plus one NaN

The existing Ragas adapter was corrected to use `AsyncOpenAI` and is covered by regression tests. That does not create retroactive scores; the semantic run must be repeated.

Those errors are not DeepEval results. DeepEval has separate adapters, columns, output paths, and error counts.

35. Ragas versus DeepEval: exact comparison

Dimension	Ragas 0.4.3 here	DeepEval 4.1.5 here
Unit	Collections metric dictionaries	LLMTestCase objects
Execution	Metric score()/ascore()	evaluate(test_cases, metrics)
Question	user_input	input
Answer	response	actual_output
Reference	reference	expected_output
Contexts	retrieved_contexts	retrieval_context
Native pass/fail	Application-owned gates	Metric threshold and success
Reason/error	MetricResult normalized	MetricData normalized
Custom policy	Custom/Ragas metric path	Policy Correctness G-Eval
Outputs	ragas_*.json/csv	deepeval_*.json/csv

The mapping is similar because the quality questions are similar. Computation is not identical because each framework owns different prompts, decomposition, wrappers, and result contracts.

36. Similarly named metrics are not equivalent

Quality question	Ragas	DeepEval	Keep beside it
Is answer grounded?	Faithfulness	Faithfulness	Claim audit and human review
Does it answer?	Answer Relevancy	Answer Relevancy	Task-specific completeness
Are useful contexts early?	Context Precision	Contextual Precision	Exact AP/nDCG
Did context cover gold?	Context Recall	Contextual Recall	Required-ID recall
How noisy is context?	Context Relevance/Utilization	Contextual Relevancy	Exact P@k and passage review
Does answer match gold?	Factual/Answer Correctness	Policy G-Eval	Concept and safety checks

Never claim Ragas 0.80 equals DeepEval 0.80. Compare trends within a pinned framework, metric version, judge, prompt configuration, and dataset. Cross-framework disagreement is a review signal, not an instruction to average.

37. What each layer can and cannot conclude

Evidence	A strong result supports	It does not establish
P@k/AP/nDCG	Approved IDs retrieved/ranked well	Answer grounded or correct
Citation validity	Cited IDs were available	Claims follow from passages
Concept coverage	Configured phrases are present	Full semantic completeness
DeepEval Faithfulness	Judge finds claims supported	Reference completeness/retrieval correctness
Contextual Recall	Expected statements supported	Exact required IDs were returned
Policy G-Eval	Answer aligns with rubric/reference	Deterministic truth or complete safety
Ragas agreement	Second framework sees same direction	Numerical equivalence/human validation

Strong release evidence triangulates exact retrieval, deterministic safety/citation checks, semantic metrics, calibrated thresholds, and periodic human adjudication.

38. Limitations: exact and deterministic metrics

- Gold IDs require human ownership and become stale when the corpus changes.
- Binary relevance hides degrees of usefulness; graded nDCG helps but labels remain subjective.
- A correct answer may use an alternative valid chunk not listed in gold.
- Regex checks can miss paraphrases or match text without understanding meaning.
- Citation IDs prove provenance membership, not claim-level entailment.
- Macro scores can hide severe failures in a safety-critical case.
- Top-k metrics do not measure freshness, access control, privacy, latency, or cost.

Mitigation: version corpus and labels together, review disagreements, preserve per-case outcomes, and keep hard safety tests.

39. Limitations: Ragas

- LLM metrics inherit judge bias, variance, language/model capability, and prompt sensitivity.
- APIs and imports changed across releases; old examples can be incompatible.
- Some metrics require references or embeddings; missing inputs change claims.
- The same generator/judge can create correlated blind spots.
- Failed metrics must remain errors; blank/NaN cells are not zero or pass.
- Context Precision can be high when one relevant item is first despite trailing noise.
- Scores do not identify document freshness, authorization, or trace-retention risk.

Use Ragas for a pinned RAG experiment suite; compare it with exact labels and human judgments instead of treating it as an oracle.

40. Limitations: DeepEval

- Thresholds are dangerous when chosen without human calibration.
- Semantic metrics depend on LLM-generated classifications and reasons.
- Local Ollama models may struggle with structured output or nuanced policy.
- G-Eval quality depends on evaluation steps and reference quality.
- Async concurrency can overload a local server or hit hosted rate limits.
- Metric reasons are diagnostic text, not proof that the score is correct.
- Caching, telemetry, dotenv behavior, and artifacts require privacy decisions.
- DeepEval does not replace exact ID-level retrieval evaluation.

Use DeepEval for test-case organization, native thresholds, custom criteria, and regression integration—while retaining exact evidence and human calibration.

41. Limitations of the current RAG itself

Current limitation	Why it matters	Production direction
Status metadata discarded	Retired/current policies can collide	Persist/filter status and dates
No stale-index detection	Edits may not rebuild	Hash corpus and build manifest
No citation entailment	Valid ID may support wrong claim	Claim-to-citation verifier
No relevance threshold	Top-k always returns evidence	Tune abstention on labels
Heading-only chunks	Long sections may be broad	Token-aware hierarchy
Dense-only retrieval	Codes/acronyms may be missed	Hybrid lexical+dense
No document ACL	Every chunk is visible	Authorization before retrieval
Full contexts in traces	Sensitive text may persist	Redaction/encryption/retention

Evaluation does not make the application production-ready. It makes known behavior measurable and regressions visible.

42. Threshold calibration and release gates

Step	Action
1	Collect representative normal, adversarial, unsupported, and high-risk cases
2	Have qualified reviewers label acceptability without seeing judge scores
3	Run a pinned judge/metric configuration repeatedly to estimate variance
4	Plot score distributions for accepted and rejected cases
5	Choose per-metric thresholds from tolerated false-pass/fail costs
6	Keep hard gates for citations, forbidden claims, and mandatory evidence
7	Version thresholds with dataset, corpus, judge, and framework
8	Recalibrate after model, corpus, prompt, or framework upgrades

The supplied 0.70 values make decisions visible during exploration. Do not enable enforced release gates until calibration is complete.

43. Recommended experiment sequence

Experiment	Keep fixed	Change	Question
A	Corpus, model, judge	top-k 1/3/5	Does recall improve before noise dominates?
B	Frozen trace/metrics	Ollama vs OpenAI judge	How stable are semantic conclusions?
C	Query/generator/judge	dense vs hybrid	Do exact IDs and context scores improve?
D	Trace/judge	core vs full	What does policy rubric add?
E	Everything	repeat run	How much judge variance exists?

```
# Run these as three explicit controlled experiments:
python deepeval_app.py run --rag-provider ollama \
  --judge-provider openai --top-k 1 --metric-profile core
python deepeval_app.py run --rag-provider ollama \
  --judge-provider openai --top-k 3 --metric-profile core
python deepeval_app.py run --rag-provider ollama \
  --judge-provider openai --top-k 5 --metric-profile core
```

Compare unique experiment IDs, not latest.csv alone. Preserve per-case values and errors; an aggregate improvement can hide a critical regression.

44. Verification evidence

Verification	Result
Fresh Python environment	Python 3.12.13
Combined install	DeepEval 4.1.5 + Ragas 0.4.3
pip check	No broken requirements
Judge adapters	OllamaModel and GPTModel initialized
Metric profiles	Core=5; full=6 including G-Eval
Compilation	All RAG/Ragas/DeepEval/CLI/test modules compile
Automated tests	22 passed
Offline CLI	Saved trace to exact JSON/CSV succeeded
Worked arithmetic	0.333, 0.500, 0.400, 0.500, 0.613 reproduced

No live semantic DeepEval quality score is claimed because the build environment had neither a running Ollama service nor an OpenAI key. Use the supplied live preflight and one-case commands.

45. Practical Q&A;: architecture and ownership

Did DeepEval replace Ragas?

No. It is a separate sibling evaluator with its own CLI, package, and reports.

Is this a second RAG?

No. Both evaluators run the existing RAGApplication or consume its saved trace.

Why reuse the same golden cases?

It controls the truth specification so framework differences are not confused with different labels.

Why keep exact metrics outside both frameworks?

Stable chunk IDs make retrieval math reproducible and independent of judge wording.

Can one trace be scored by both?

Yes. Preserve the trace and run each evaluator separately with the same gold case.

Which report is authoritative?

JSON is complete; CSV is a flattened analysis view.

46. Practical Q&A:: metrics

Why can Hit Rate be 1.0 while recall is poor?

Every query found one relevant item, but additional mandatory items were missed.

Why can Faithfulness be high while correctness is low?

The answer can accurately repeat incomplete or wrong retrieved context.

Is Contextual Precision the same as P@k?

No. DeepEval judges semantic relevance; P@k uses exact labels.

Does Contextual Recall replace required-ID recall?

No. One measures semantic reference coverage; the other checks exact mandatory sources.

Should Ragas and DeepEval scores be averaged?

No. Use agreement/disagreement as evidence; compare trends within each framework.

Does a judge reason prove the score?

No. It is diagnostic output from the same fallible judge process.

47. Practical Q&A;: operation and failures

When should I use `--skip-deepeval`?

For offline checks, fast CI, dataset validation, retrieval experiments, and preflight diagnostics.

Does `--skip-deepeval` pass semantic quality?

No. It records no semantic attempts and supports no semantic conclusion.

What does `--allow-metric-errors` do?

It changes process exit policy; it never repairs an error or creates a score.

What should I do with a blank score?

Read the paired error. Blank plus error means no score was produced.

Why start with one case?

It validates connectivity, structured output, latency, report shape, and cost before eight cases.

Why use an independent judge?

It can reduce correlated blind spots, though it adds another model and calibration need.

48. Practical Q&A:: change decisions

When should I change top-k?

When exact recall and context noise show a retrieval-depth tradeoff under controlled comparison.

When should I change the retriever?

When labeled required IDs are consistently missed, especially after top-k calibration.

When should I change the prompt?

When retrieval is adequate but answers omit, distort, overstate, or cite poorly.

When should I change the judge?

When structured output fails, human agreement is weak, or repeated scores are unstable.

When should I change a threshold?

Only after human calibration shows unacceptable false passes or false failures.

When is a release defensible?

When hard controls pass, semantic gates are calibrated, errors are absent, and high-risk cases receive review.

49. Execution checklist

Before the run

- Create a fresh environment and install pinned requirements.
- Keep the real API key only in local .env; never archive it.
- Run pip check and all 22 tests.
- Review the golden case and required IDs being evaluated.
- Run non-live preflight, then one live structured-output preflight.

After the run

- Confirm intended RAG and judge providers/models.
- Confirm expected semantic scores are numeric and errors blank.
- Inspect exact retrieval/citation before aggregate judge means.
- Read failed answers, contexts, reasons, and gold together.
- Do not use default thresholds as release gates until calibrated.
- Preserve the unique experiment files: do not rely only on latest.

A report is useful when it supports a specific engineering decision. If it cannot distinguish retrieval, generation, evaluation, and data failures, evidence was aggregated too early.

50. Command reference

```
# Application only
python app.py build --provider ollama
python app.py ask "<question>" --provider ollama --top-k 3 --show-context

# Cases and preflight
python deepeval_app.py list-cases
python deepeval_app.py preflight --judge-provider ollama --live

# One live case, then all cases
python deepeval_app.py run --rag-provider ollama \
  --judge-provider ollama --case-id MFA-001 --top-k 3 \
  --metric-profile core --show-reasons
python deepeval_app.py run --rag-provider ollama \
  --judge-provider openai --top-k 3 --metric-profile full

# Frozen trace and exact-only paths
python deepeval_app.py trace --trace results/latest.json \
  --case-id MFA-001 --judge-provider openai --metric-profile core
python deepeval_app.py trace \
  --trace deepeval_evaluation/data/sample_mfa001_trace.json \
  --case-id MFA-001 --skip-deepeval

# Existing Ragas remains independent
python eval_app.py run --rag-provider ollama \
  --judge-provider openai --top-k 3 --metric-profile core
```

Use README-DEEPEVAL.md for platform-specific setup and complete option explanations.

51. Source-to-evidence map

Question	Source to inspect
What did the RAG retrieve?	results/.json and rag/index.py
What answer did it generate?	Trace answer and rag/pipeline.py
What truth was used?	evaluation/data/golden_cases.json
How were exact scores calculated?	evaluation/retrieval_metrics.py
How were citations/concepts checked?	evaluation/deterministic_metrics.py
How was LLMTestCase built?	deepeval_evaluation/adapters.py
Which judge/model ran?	deepeval_evaluation/judges.py plus report JSON
Which metrics/thresholds ran?	deepeval_evaluation/metrics.py plus report JSON
How were errors aggregated?	deepeval_evaluation/reporting.py
How do I reproduce commands?	README-DEEPEVAL.md and deepeval_app.py
How is Ragas separate?	evaluation/ragas_runner.py and eval_app.py

52. Authoritative references

Framework/API behavior was checked against official DeepEval, Ragas, OpenAI, Ollama, and Stanford IR sources. Project behavior comes from the supplied source and tests.

- [D1] [DeepEval 4.1.5 package](https://pypi.org/project/deepeval/)
<https://pypi.org/project/deepeval/>
- [D2] [DeepEval getting started](https://deepeval.com/docs/getting-started)
<https://deepeval.com/docs/getting-started>
- [D3] [DeepEval metrics introduction](https://deepeval.com/docs/metrics-introduction)
<https://deepeval.com/docs/metrics-introduction>
- [D4] [DeepEval Answer Relevancy](https://deepeval.com/docs/metrics-answer-relevancy)
<https://deepeval.com/docs/metrics-answer-relevancy>
- [D5] [DeepEval Faithfulness](https://deepeval.com/docs/metrics-faithfulness)
<https://deepeval.com/docs/metrics-faithfulness>
- [D6] [DeepEval Contextual Precision](https://deepeval.com/docs/metrics-contextual-precision)
<https://deepeval.com/docs/metrics-contextual-precision>
- [D7] [DeepEval Contextual Recall](https://deepeval.com/docs/metrics-contextual-recall)
<https://deepeval.com/docs/metrics-contextual-recall>
- [D8] [DeepEval Contextual Relevancy](https://deepeval.com/docs/metrics-contextual-relevancy)
<https://deepeval.com/docs/metrics-contextual-relevancy>
- [D9] [DeepEval evaluation datasets](https://deepeval.com/docs/evaluation-datasets)
<https://deepeval.com/docs/evaluation-datasets>
- [D10] [DeepEval environment variables](https://deepeval.com/docs/environment-variables)
<https://deepeval.com/docs/environment-variables>
- [D11] [DeepEval Ollama integration](https://deepeval.com/integrations/models/ollama)
<https://deepeval.com/integrations/models/ollama>
- [D12] [DeepEval OpenAI integration](https://deepeval.com/integrations/models/openai)
<https://deepeval.com/integrations/models/openai>
- [R1] [Ragas 0.4.3 package](https://pypi.org/project/ragas/)
<https://pypi.org/project/ragas/>
- [R2] [Ragas v0.4 migration](https://docs.ragas.io/en/stable/howtos/migrations/migrate_from_v03_to_v04/)
https://docs.ragas.io/en/stable/howtos/migrations/migrate_from_v03_to_v04/
- [R3] [Ragas Faithfulness](https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/faithfulness/)
https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/faithfulness/
- [R4] [Ragas Context Precision](https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_precision/)
https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_precision/
- [R5] [Ragas Context Recall](https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_recall/)
https://docs.ragas.io/en/stable/concepts/metrics/available_metrics/context_recall/
- [O1] [OpenAI API quickstart](https://developers.openai.com/api/docs/quickstart)
<https://developers.openai.com/api/docs/quickstart>
- [O2] [Ollama Python client 0.6.2](https://pypi.org/project/ollama/)
<https://pypi.org/project/ollama/>
- [I1] [Stanford IR evaluation chapter](https://nlp.stanford.edu/IR-book/html/htmledition/evaluation-in-information-retrieval-1.html)
<https://nlp.stanford.edu/IR-book/html/htmledition/evaluation-in-information-retrieval-1.html>